bcc09 for Vectrex Programmers

C on the Vectrex, for those who know its BIOS and 6809 assembly

bcc09 project

November 2026

Contents

1 Why C? **2**

2 Your first cartridge **2**

3 The header **4**

4 C for the assembly programmer **4**

5 The BIOS from C **6**

6 A frame **7**

7 Controls and sound **8**

8 Memory **9**

9 Speed **10**

10 Assembly with C **11**

11 A complete game **12**

12 Testing and debugging **12**

Appendix: for the assembly programmer **12**

1 Why C?

If you have written a Vectrex game in assembly, you know the machine: the BIOS's routines by the names in `VECTREX.INC`, `Wait_Recal` at the top of every frame, DP at `$D0` for the VIA, the RAM from `$C880` that the BIOS leaves you. *bcc09* lets you write the same game in C, calling the same routines by the same names:

```
#include <vectrex.h>

int main()
{
    for (;;) {
        Wait_Recal();
        Intensity_a(0x7F);
        Print_Str_d(0, -40, "HELLO\200");
    }
}
```

- **The game's logic is easier to write and to change.** Structures for the things on the screen, loops over them, functions for each part of a frame: the bookkeeping that takes pages of assembly takes lines of C.
- **The BIOS is the same BIOS.** Each function in `vectrex.h` puts its arguments where the routine wants them, sets DP, calls it, and puts your DP back. What you know about the routines still holds.
- **Assembly is still there** for what needs it: a module of your own, linked with the C, or a few instructions in a C function.

What it costs is some speed and some ROM: a C function call does more than a `JSR`, and the compiler doesn't keep values in registers as you would. For the logic of most games that doesn't matter; for the inner loop of a drawing routine it may, and that is what assembly is for.

This guide is for programmers who know the Vectrex and 6809 assembly, and some C. It shows how bcc09 builds a cartridge, how C maps onto what you know, and what to watch for. The bcc09 user's manual has the rest of the compiler, and *bcc09 for BASIC Programmers* teaches C itself.

1.1 What you need

- **bcc09**, on a PC running Linux: on Fedora, the RPM from bcc09's releases on GitHub; otherwise, built from its source.
- **An emulator:** MAME's `vectrex` runs bcc09's cartridges as they are, with the BIOS image (`exec_rom.bin`). Any emulator that takes a cartridge's `.bin` will do.
- **For a real Vectrex,** a flash cartridge, or an EPROM: the output is the ROM's image, from `$0000`.

2 Your first cartridge

Save the program above as `hello.c`, and:

```
bcc09 -Mv -O -o hello.bin hello.c
mame vectrex -cart hello.bin
```

`-Mv` is the Vectrex target: the compiler, the start-up code, the library and the layout of a cartridge. `-O` runs the peephole optimizer, which makes the code smaller and faster. `hello.bin` is the cartridge's image, 319 bytes (a program that does nothing is 144); MAME shows the BIOS's title screen, with the default title "BCC09" and the BIOS's tune, and then runs the program.

2.1 What's in the cartridge

Address	What
`$0000`	the header: `g GCE`, the year, the music, the title
after it	the start-up code, which the BIOS runs after the header
	the program's code, its constants, and an image of its initialized data
`$C800-$C87F`	RAM: the BIOS's variables
`$C880`	RAM: the program's data, then its uninitialized data (BSS)
below `$CBEA`	RAM: the stack, growing down
`$D000`	the VIA

The start-up code (`crtvec`) sets S to `$CBEA` and DP to `$D0` (`$C8` with `__direct` variables: section 4.4), copies the initialized data from the ROM to the RAM, clears the BSS, and calls `main`. `main` should never return: if it does, the program stops in a loop.

The cartridge can be up to 32K. The RAM is the hard limit: 874 bytes for the data, the BSS and the stack together. Chapter 8 is about making it last.

2.2 Seeing what the compiler did

`-S` stops after the compiler and writes the assembly, which is worth reading when you want to know what a line of C costs:

```
bcc09 -Mv -O -S -o hello.s hello.c
```

`-X-M` asks the linker for a map, with every symbol's address: what you need to set a breakpoint in MAME's debugger (`mame vectrex -cart hello.bin -debug`) or to watch a variable.

```
bcc09 -Mv -O -o hello.bin -X-M hello.c > hello.map
```

A C name is the assembler's with an underscore before it: `main` is `_main`, a variable `score` is `_score`.

3 The header

Unless the program has its own, the header is the library's: "BCC09", with the BIOS's tune. A program's own header is a few lines of assembly that define `__vec_header` in segment 12, which the linker puts at `$0000`. In C, it's `asm()` outside a function:

```
#include <vectrex.h>

asm("loc 12", "export __vec_header", "__vec_header:",
    "fcc \"g GCE 2026\"", "fcb $80",    /* "g GCE", the year */
    "fdb $FD0D",            /* the music: Music1 */
    "fcb $F8,$50,$20,$D0",      /* height, width, y, x */
    "fcc \"MY GAME\"", "fcb $80,0");    /* the title; the end */

int main()
{
    for (;;)
        Wait_Recal();
}
```

The fields are the BIOS's: the copyright string (the BIOS checks for `g GCE`, and shows the year after it), the address of the music it plays on the title screen (the BIOS's own tunes are `$FD0D` and the others `vectrex.h` names `Music1` to `MusicD`; or one of your own, in the cartridge), and then the title, as lines of height, width, y, x and text ended by `$80`, the last followed by 0.

The same lines can be a file of assembly instead, linked with the program:

```
bcc09 -Mv -O -o game.bin game.c title.s
```

4 C for the assembly programmer

4.1 The types

C	Size	In assembly
`char`, `unsigned char`	8 bits, unsigned	`LDB`, `CLRA` to widen
`signed char`	8 bits, signed	`LDB`, `SEX` to widen
`int`, `short`	16 bits, signed	`LDD`
`unsigned`	16 bits	`LDD`
`long`	32 bits	a library call for most operations
pointers	16 bits	`LDX`

A plain `char` is unsigned. That matters on the Vectrex, where coordinates are signed bytes: a list of lines, a position, a joystick's reading are `signed char`. A `char` holding -10 is 246 to C: `if (dy < 0)` is never true, and `y + dy` adds 246, not -10. `vectrex.h` declares the lists as `signed char *` and the joysticks' variables as signed; declare your own the same way.

`float` and `double` work, in software, but a multiplication takes thousands of cycles and the code is several K: not for a game's frame. Use fixed point (chapter 9).

4.2 Variables and registers

A function's local variables are on the stack, addressed from S; global and `static` ones are at fixed addresses in the RAM, reached with extended addressing (`LDD >_score`). The compiler loads a value when it needs it and stores it back, rather than keeping it in a register across statements as you would. This, from `-O`,

```
struct thing { char size; int y, x, vy, vx; };

void move(t)
struct thing *t;
{
    t->y += t->vy;
    t->x += t->vx;
}
```

becomes

```
_move:  PSHS    X               ; t, the first argument
        LDX     0,S
        LDU     0,S
        LDD     1,U             ; t->y
        ADDD    5,X             ; + t->vy
        STD     1,U
        LDX     0,S
        LDU     0,S
        LDD     3,U             ; t->x
        ADDD    7,X             ; + t->vx
        STD     3,U
        PULS    Y,PC            ; (Y: just to drop t)
```

which is what you would write, with a few loads more. A loop over an array of structures through a pointer, as here, is about as good as C gets on the 6809: indexing with `rocks[i]` multiplies `i` by the structure's size each time.

4.3 Functions

The first argument is passed in X, the rest on the stack (the caller removes them), and the result comes back in X (D and X for a `long`). A function may change every register but DP. A call costs a `JSR` and the pushes and pulls around it: a few dozen cycles. Chapter 10 has the details for writing functions in assembly.

4.4 The direct page

The 6809's direct page makes a variable's every use a byte smaller and a cycle faster, and the compiler uses it for variables declared `__direct`:

```
__direct int frame;
__direct unsigned char lives;
```

`++lives` is then `LDB <_lives`, `INCB`, `STB <_lives`. The BIOS's variables are in page `$C8`, and the program's `__direct` variables go after them, at `$C880`, with DP set to `$C8` while the program runs (the functions in `vectrex.h` still give the BIOS `$D0`, and `$C8` for the music, and put the program's back after). That leaves room for 128 bytes of them (a little less, as the linker rounds each module's up to a multiple of 4); if there are more, they all move to the next page, `$C9`, and the 128 bytes after the BIOS's variables go unused. A program without `__direct` variables keeps DP at `$D0`.

5 The BIOS from C

`vectrex.h` declares a function for each BIOS routine bcc09 supports, with the routine's name, and the BIOS's variables (`Vec_Btn_State`, `Vec_Joy_1_X`, `VIA_t1_cnt_lo`, `Vec_Loop_Count` and the others), which are the bytes in the RAM themselves: `Vec_Music_Flag = 1;` stores 1 at `$C856`.

Each function takes what the routine takes in its registers, in the order the routine's documentation gives them, y before x:

Routine	From C	The registers
`Wait_Recal`	`Wait_Recal()`	
`Intensity_a`	`Intensity_a(i)`	A = i
`Reset0Ref`	`Reset0Ref()`	
`Moveto_d`	`Moveto_d(y, x)`	A = y, B = x
`Draw_Line_d`	`Draw_Line_d(y, x)`	A = y, B = x
`Dot_d`	`Dot_d(y, x)`	A = y, B = x
`Draw_VLc`	`Draw_VLc(list)`	X = list
`Draw_VL_a`	`Draw_VL_a(n, list)`	A = n, X = list
`Mov_Draw_VL_a`	`Mov_Draw_VL_a(n, list)`	A = n, X = list
`Draw_VLp`	`Draw_VLp(list)`	X = list
`Draw_Pat_VL_a`	`Draw_Pat_VL_a(n, pattern, list)`	A = n, `Vec_Pattern`, X = list
`Dot_List`	`Dot_List(n, list)`	`Vec_Misc_Count` = n, X = list
`Rot_VL_ab`	`Rot_VL_ab(angle, n, from, to)`	A = angle, B = n, X, U
`Print_Str_d`	`Print_Str_d(y, x, s)`	A = y, B = x, U = s
`Print_Ships`	`Print_Ships(ship, n, y, x)`	A = ship, B = n, X = y, x
`Read_Btns`	`b = Read_Btns()`	the result: A
`Joy_Digital`	`Joy_Digital()`	
`Init_Music_chk`	`Init_Music_chk(tune)`	U = tune, DP `$C8`

Routine	From C	The registers
`Explosion_Snd`	`Explosion_Snd(parms)`	U = parms, DP `$C8`
`Random`	`r = Random()`	the result: A

and `Intensity_1F` to `_7F`, `Moveto_d_7F`, `Draw_VLcs`, `Print_Str_yx`, `Print_List`, `Clear_Score`, `Add_Score_a`, `Add_Score_d`, `New_High_Score`, `Read_Btns_Mask`, `Joy_Analog`, `Select_Game`, `Do_Sound`, `Sound_Byte`, `Sound_Bytes` and `Clear_Sound`; `vectrex.h` says what each takes. Counts are the BIOS's: where it draws "n + 1 lines", n is one less than the lines, as in the routine.

Strings for the BIOS are in capitals and end with `$80`, which C writes `\200`: `"GAME OVER\200"`. A C string's own 0 comes after it, unused.

5.1 What a call costs

A function in `vectrex.h` saves DP, sets it, moves its arguments into the registers, calls the routine and restores DP. Counted by the instructions, `Moveto_d(y, x)` from C takes about 90 cycles more than `LDD`, `JSR Moveto_d` in assembly: about 60 in the library's function, and 30 in the C call (pushing x, loading y, the `JSR` and dropping x). The BIOS's frame is 30,000 cycles (50 a second): a few dozen calls a frame cost little, a few hundred a good part of it. Drawing a shape as one list (`Draw_VLc`, `Mov_Draw_VL_a`) rather than a call for each line saves the most.

5.2 Routines vectrex.h doesn't have

Call them from assembly (chapter 10): a function of your own that does what the ones in the library do, `PSHS DP`, the DP the routine wants, the registers, `JSR`, `PULS DP,PC`.

6 A frame

A frame is what you know from assembly, in C:

```
#include <vectrex.h>

signed char box[] = { 3, 0, 40, 40, 0, 0, -40, -40, 0 };

int main()
{
    for (;;) {
        Wait_Recal();       /* the frame starts */
        Intensity_a(0x7F);
        Reset0Ref();        /* the beam to the centre */
        VIA_t1_cnt_lo = 0x7F;   /* the scale of moves */
        Moveto_d(-20, -20);
        VIA_t1_cnt_lo = 0x40;   /* and of the lines */
        Draw_VLc(box);      /* 3: four lines */
```

```
    }
}
```

- **The scale** is `VIA_t1_cnt_lo`: set it before moving and before drawing, as in assembly. A list at a smaller scale is a smaller shape, the same list.
- **Positions** after `Reset0Ref()` are from the centre, -128 to 127 at the scale of the move; drift builds up over moves, so start each object from the centre.
- **Text** (`Print_Str_d`) is at y, x from the centre; `Vec_Text_HW` holds its height and width.
- **Rotation:** `Rot_VL_ab(angle, n, from, to)` turns n + 1 points (a list's pairs) by angle, 64 to a turn, anticlockwise, into another array (`to` may be `from`); a list's lines rotate as its points do.

7 Controls and sound

7.1 The joysticks and the buttons

```
#include <vectrex.h>

int main()
{
    Vec_Joy_Mux_1_X = 1;    /* read joystick 1's X and Y */
    Vec_Joy_Mux_1_Y = 3;
    Vec_Joy_Mux_2_X = 0;    /* not joystick 2 */
    Vec_Joy_Mux_2_Y = 0;
    for (;;) {
        Wait_Recal();
        Joy_Digital();
        Read_Btns();        /* (it sets Vec_Btn_State too) */
        if (Vec_Btn_State & 0x08)   /* button 4 held down */
            Intensity_a(0x7F);
        else
            Intensity_a(0x3F);
        Reset0Ref();        /* a dot where the stick points */
        Dot_d(Vec_Joy_1_Y < 0 ? -40 : Vec_Joy_1_Y > 0 ? 40 : 0,
              Vec_Joy_1_X < 0 ? -40 : Vec_Joy_1_X > 0 ? 40 : 0);
    }
}
```

- **`Joy_Digital()`** reads the joysticks the mux variables enable into `Vec_Joy_1_X` and the others: below 0, 0 or above 0. `Joy_Analog()` gives how far, -128 to 127; `Vec_Misc_Count` must be 0 before it.
- **`Read_Btns()`** gives a bit for each button pressed since the last call (bits 0 to 3 controller 1's buttons 1 to 4, bits 4 to 7 controller 2's); `Vec_Btn_State` has those held down. `Read_Btns_Mask(mask)` lets a button whose bit is clear in mask count again while it's held.
- **`Select_Game(players, games)`** shows the BIOS's screen for choosing the number of players and the game, which ends after a while with no button pressed; the choices are in `Vec_Num_Players` and `Vec_Num_Game`, from 1.

7.2 Music and sound

The BIOS plays music a frame at a time: set `Vec_Music_Flag` to 1, and in each frame call `Init_Music_chk(tune)` before `Wait_Recal()` and `Do_Sound()` after it, until `Vec_Music_Flag` is 0 again.

```
#include <vectrex.h>

int main()
{
    Vec_Music_Flag = 1;
    for (;;) {
        Init_Music_chk(Music2);
        Wait_Recal();
        Do_Sound();
        Intensity_a(0x7F);
        Print_Str_d(0, -60, "MUSIC\200");
        if (!Vec_Music_Flag)    /* the tune has ended: again */
            Vec_Music_Flag = 1;
    }
}
```

An explosion is the same: `Vec_Expl_Flag = 0x80` starts one, and `Explosion_Snd(parms)` (four bytes of the BIOS's parameters) goes in each frame, before `Do_Sound()`. `Sound_Byte(reg, value)` and `Sound_Bytes(list)` write the sound chip's registers themselves. `Clear_Sound()` quiets it; the BIOS's version sets registers 0 to 13 to 1 rather than 0 (near silence) and turns the music buffer's mixer off, so it is silent after the next `Do_Sound()`.

8 Memory

The RAM is 874 bytes, from `$C880` to `$CBEA`, for everything: the data, the BSS, and the stack. The map (`-X-M`) shows how much the data and BSS take: `__end` is where they end, and the stack has the rest.

8.1 Constant tables belong in the ROM

bcc09's compiler doesn't know `const`, so an initialized array is data:

```
signed char ship[] = { 2, -16, -7, 0, 14, 16, -7 }; /* 3 lines */
```

takes 7 bytes of ROM, for its image, and 7 bytes of the RAM, where the start-up code copies it. For a few shapes that's fine. For big tables (music, many shapes, level maps), put them in the ROM with `asm()`, and declare them `extern`:

```
#include <vectrex.h>

asm("loc 0", "export _ship", "_ship:",
    "fcb 2,-16,-7,0,14,16,-7");
extern signed char ship[];

int main()
{
    for (;;) {
        Wait_Recal();
        Intensity_a(0x7F);
        Reset0Ref();
        VIA_t1_cnt_lo = 0x40;
        Draw_VLc(ship);
    }
}
```

`loc 0` is the code's segment, in the ROM. (The compiler says which segment it is in again after assembly code outside a function, so the data after it still goes where it should.) String constants are in the ROM already: `Print_Str_d(0, 0, "HELLO\200")` uses no RAM for the text.

8.2 The stack

Local variables, arguments and return addresses are on the stack, which grows down from `$CBEA` towards the data. Nothing checks that it doesn't reach it: keep big arrays out of functions (make them `static`, or global, where the map shows their size), and recursion shallow.

8.3 The heap

`malloc()` works, between the end of the BSS and 256 bytes below the stack pointer, which on the Vectrex leaves little or nothing. Allocate arrays of what the game needs instead.

9 Speed

- **Fixed point instead of floating point.** Keep a position in an `int` as 8.8: the high byte is the screen's coordinate, the low byte the fraction. `y += vy` moves it by a fraction of a point a frame, `y >> 8` is the coordinate, and a position that runs off one side comes back on the other by itself, as the `int` wraps.
- **Multiplication by constants.** The compiler multiplies by a constant with shifts and adds (`x * 3` is `x + (x << 1)`), but `x * y` is a call into the library's 16-bit multiplication, and division by anything but a power of two (`x / 3`) a call into its division, slower still. A signed `x / 4` corrects for a negative `x` first; `x >> 2` doesn't.
- **`unsigned char`** for counters and flags that fit in a byte, and **`__direct`** for the variables a frame uses most.
- **Pointers** to walk arrays of structures, as in section 4.2, rather than indexes.

- **Lists**, for drawing: one call for a whole shape.
- **Read the output** (`-S`) of the code a frame runs most; and if a routine must be fast, write it in assembly.

10 Assembly with C

10.1 A module of your own

A function in assembly takes its first argument in X and the rest on the stack above the return address, and returns its result in X:

```
; int twice(int n): n * 2
        .text
        export  _twice
_twice: tfr     x,d             ; n, the first argument
        lslb
        rola
        tfr     d,x             ; the result, in X
        rts
```

It may change every register but DP, which it must leave as it found it. Assembled and linked with the program (`bcc09 -Mv -O -o game.bin game.c twice.s`), it's called from C as `twice(21)`, after a declaration `int twice(int n);`.

A routine that calls the BIOS sets DP itself, and puts the program's back:

```
; void flash(int y, int x): a dot at y, x, at full brightness
        .text
        export  _flash
_flash: pshs    dp              ; the program's DP
        lda     #$D0            ; the BIOS's
        tfr     a,dp
        tfr     x,d             ; y, the first argument
        tfr     b,a
        ldb     4,s             ; x (above DP and the return address)
        pshs    d
        jsr     $F2A9           ; Intensity_7F
        jsr     $F354           ; Reset0Ref
        puls    d
        jsr     $F2C3           ; Dot_d
        puls    dp,pc
```

10.2 Assembly inside a function

`asm()` in a function puts instructions in the code where it is:

```
__direct unsigned char ticks;

void tick()
{
    asm("inc <_ticks");
}
```

The instructions can use global variables by their assembler names (`_score`), but not local ones, whose places on the stack the compiler decides; and they must leave DP and S as they found them.

11 A complete game

`doc/rocks.c` (installed with bcc09's documentation) is a small game, about 300 lines: rocks drift across the screen, and the ship turns, thrusts and fires at them. It makes a 2K cartridge, uses 234 bytes of the RAM, and shows most of this guide at work:

- its header in C, with the title "ROCKS";
- the things on the screen as an array of structures, with positions and speeds in 8.8 fixed point;
- the ship turned with `Rot_VL_ab` each frame, and drawn as a list with `Mov_Draw_VL_a`; the rocks as one list, at two scales;
- the joystick and the buttons, `Select_Game`, explosions' sounds;
- the score and the high score with the BIOS's score routines, the ships left with `Print_Ships`.

It is under the ISC license, not the GPL: start a game of your own from it.

```
bcc09 -Mv -O -o rocks.bin rocks.c
mame vectrex -cart rocks.bin
```

12 Testing and debugging

- **MAME's debugger** (`-debug`) steps through the code, with the map's addresses for breakpoints; `-X-M` writes the map.
- **MAME's Lua scripts** (`-autoboot_script`) can read the RAM and press the buttons and move the joystick each frame, for tests that play a game by themselves: bcc09's own tests play `rocks.c` that way, to the same score every time.
- **The logic away from the BIOS:** what doesn't draw (the game's rules, its arithmetic) can be compiled for sim6809 with `-Mi`, or for your PC with its own C compiler, and tested there.

Appendix: for the assembly programmer

In assembly	In C
`jsr Wait_Recal`	`Wait_Recal();`
`lda #$7F` / `jsr Intensity_a`	`Intensity_a(0x7F);`
`ldd #$1020` / `jsr Moveto_d`	`Moveto_d(0x10, 0x20);`
`ldx #ship` / `jsr Draw_VLc`	`Draw_VLc(ship);`
`lda #$7F` / `sta VIA_t1_cnt_lo`	`VIA_t1_cnt_lo = 0x7F;`
`lda Vec_Joy_1_X`	`Vec_Joy_1_X`
`fcb $80` ending a string	`\200` at the end of a C string
`fcb -10`	`signed char` (a plain `char` is unsigned)
`ldd >score`	`score`
`ldd <score` (DP)	`score`, declared `__direct`
`rmb 10`	`char buf[10];` (in the BSS)
a table in the ROM	`asm("loc 0", ...)` and `extern`
`bsr sub`	`sub();`
`lsla`	`<< 1`
`asra`	`>> 1` (of a signed value)